# Trade-offs in Partition Assignment for Distributed Dynamic Graph Embedding

Stefan Nožinić

## Introduction

A graph is a mathematical structure consisting of vertices (or nodes) connected by edges. Graphs are widely used to model relationships and interactions in various domains (Van Der Hofstad 2024), such as social networks (Leskovec et al. 2010b) (Backstrom et al. 2006) (Rozemberczki and Sarkar 2021), collaboration networks (Savić et al. 2017), terrorist networks (Krebs 2002) and blog citation networks (Adamic and Glance 2005). In these applications, the relationships between entities can be represented as edges connecting the corresponding vertices.

In many real-world graphs, the degree distribution follows a power-law, meaning that a small number of vertices have a very high degree (i.e., they are connected to many other vertices), while most vertices have a low degree. This characteristic is often observed in social networks, where a few individuals (e.g., celebrities) have many connections, while the majority of users have relatively few connections.

In real graphs, there are several properties that are often observed (Watts and Strogatz 1998) (Zachary 1977) (Albert and Barabási 2002): - **Small-world property**: Most pairs of vertices can be connected by a short path, even in large graphs. This is often referred to as the "six degrees of separation" phenomenon. (Watts and Strogatz 1998) - **Community structure**: Vertices tend to form clusters or communities, where vertices within the same community are more densely connected than those in different communities. This property is prevalent in social networks, where groups of friends or colleagues often form tightly-knit communities. (Leskovec et al. 2009) - **Scale-free property**: The degree distribution of the graph follows a power-law, meaning that a few vertices have a very high degree, while most vertices have a low degree. This is often observed in social networks, where a small number of individuals (e.g., celebrities) have many connections, while the majority of users have relatively few connections. (Barabási and Albert 1999)

Graph vertex embeddings are a powerful technique for representing vertices in a graph as low-dimensional vectors, enabling various machine learning tasks such as vertex classification, link prediction (Leskovec et al. 2010a), and community detection. The effectiveness of these embeddings often depends on the underlying

graph structure and the methods used to generate them. When faced with large graphs, the challenge of efficiently computing these embeddings while preserving the graph's structural properties becomes paramount. Additionally, most real-world graphs are dynamic, with vertices and edges being added or removed over time. This dynamic nature introduces additional challenges in maintaining accurate and up-to-date embeddings, as the graph's structure evolves.

This leads to a practical trade-off between early partition assignment (low latency) and informed partition assignment (higher embedding quality and partition balance). Despite its practical relevance, this trade-off has not been systematically studied in the context of dynamic graph embedding.

# Problem Formulation

Given a dynamic graph where nodes arrive online and a distributed dynamic Node2Vec-style embedding model is maintained, we want to know how should partitions be assigned to new nodes to balance:

- embedding quality,
- partition balance,
- assignment latency

This paper aims to empirically study and characterize the trade-offs between delayed and immediate partition assignment strategies for online node arrivals in distributed dynamic graph embedding systems. Rather than proposing a new embedding model, the focus is on partition assignment heuristics and their systemic impact. In this paper, existing embedding model is used which handles dynamic nature of the evolving graph.

## Related work

Graph vertex embedding is a well-studied area, with various methods proposed to generate low-dimensional representations of nodes in a graph. State of the art method which is widely used is Node2Vec (Grover and Leskovec 2016) which uses random walks to capture the local and global structure of the graph. Node2Vec generates embeddings by performing biased random walks on the graph, allowing it to explore both local and global structures. The method has been shown to be effective in capturing community structures and generating meaningful embeddings for various machine learning tasks. As its improvement, DistGER (Fang et al. 2023) is a distributed graph embedding method that extends Node2Vec by leveraging distributed computing to handle large graphs. DistGER uses a similar random walk approach but optimizes walk sampling in order to maximize the information gain when selecting the next vertex to visit.

Another approach to scale Node2Vec is proposed in (Lombardo and Poggi 2019) which is based on actor model and uses a distributed framework to generate embeddings for large graphs. This method allows for parallel processing of

random walks, significantly improving the efficiency of embedding generation in terms of time and resource usage.

The common ground for these methods is that they generate walks which are later used to train Word2Vec model (Church 2017) commonly used for generating embeddings in natural language processing tasks.

During the learning process, there are several state of the art approaches to parallelize the training of word2vec model. Commonly used approach in distributed environment is ensemble learning (Ji and Ling 2007) which combines multiple smaller models to create a larger model. Final mode is created by aggregating smaller models using parameter server architecture (Li et al. 2013).

The main challenge to address the problem of distributed graph vertex embedding is partitioning the graph in a way that preserves the community structure while ensuring that the partitions are balanced and can be processed efficiently in a distributed environment. Up until recently, most partitioning methods covered only small graphs or graphs without inherent community structure, like in (Benlic and Hao 2010) (Sanders and Schulz 2012) (Sanders and Schulz 2011) (Romero Ruiz and Segura 2018) (Çatalyürek et al. 2023). The main focus of these methods is static graph partitioning meaning that the graph is partitioned once and then used for processing. However, in many real-world applications, graphs are dynamic and change over time, requiring dynamic partitioning methods that can adapt to changes in the graph structure. In (Ugander and Backstrom 2013), a variant of label propagation algorithm is proposed for balancing partitions, but it lacks the ability to adapt to changes in the graph structure over time and it does not study the impact of partitioning on embedding quality.

For dynamic graph partitioning, there are several methods available in literature like (Nicoara et al. 2015) (Huang and Abadi 2016) (Xu et al. 2014) and (Vaquero et al. 2013). These methods focus on partitioning dynamic graphs by considering the changes in the graph structure over time and adapting the partitioning accordingly. However, these methods have not been used in embedding applications so far, and their effectiveness in generating high-quality embeddings in a distributed environment remains an open question. However, temporal graph embedding methods like (Mahdavi et al. 2018) have been proposed to address the problem of dynamic graph embedding. These methods focus on generating embeddings for dynamic graphs by considering the temporal evolution of the graph structure. However, these methods do not address the problem of partitioning the graph in a distributed environment, which is crucial for efficient processing and scalability.

Distributed graph embedding methods such as dynamic Node2Vec (Lombardo and Poggi 2019) (Mahdavi et al. 2018) variants enable scalable representation learning on large, evolving graphs. However, in dynamic settings with online node arrivals, a fundamental systems problem arises: how to assign newly arriving nodes to partitions before sufficient structural or embedding information is available.

Most existing distributed embedding systems assume either:

- static partitioning (Benlic and Hao 2010) (Sanders and Schulz 2012), or
- immediate assignment based on incomplete information (Ugander and Backstrom 2013).

# Contributions

This paper makes the following contributions:

- We propose a systematic study of the trade-offs between delayed and immediate partition assignment strategies for online node arrivals in distributed dynamic graph embedding systems.
- We evaluate the impact of different partition assignment strategies on embedding quality, partition balance, and assignment latency, providing insights into the practical implications of these strategies in real-world scenarios.
- We provide empirical evidence and analysis of the trade-offs involved in partition assignment strategies, contributing to the understanding of how to effectively manage partitioning in distributed dynamic graph embedding systems.

## Paper organization

The rest of the paper is organized as follows: First, system overview is presented, then graph partitioning and embedding model are described. Next, benchmarks are explained in detail. Finally, results and discussion are presented, followed by the conclusion and references.

# Methods

## System overview

Temporal graph is represented as a sequence of events. Each event is a tuple (t, u, v) where t is the timestamp of the event and u and v are the vertices involved in the event. The events are processed in chronological order, and the graph is updated accordingly. The system maintains a distributed dynamic Node2Vec-style embedding model, which is updated as new events arrive. The partition assignment strategy determines how new vertices are assigned to partitions in the distributed system.

The system buffers incoming events for a short period before assigning them to partitions. This buffering allows the system to make more informed partitioning decisions by considering a batch of events together, rather than assigning partitions immediately upon the arrival of each event. Once the events are buffered,

the partition assignment strategy is applied, and the embeddings are updated accordingly.

In the following listing, pseudocode for the system is presented:

```
global state buffer = {}

when new event (t, u, v) arrives:
    buffer.add((t, u, v))
    if buffer.size() >= BUFFER_SIZE:
        assign_partitions(buffer)
        update_embeddings(buffer)
        buffer.clear()
```

Buffer also defines new graph snapshot. Let $G_n$ be the graph snapshot after processing the first $n$ buffers. Now, let $B_n$ be the $n$-th buffer of events. The graph snapshot $G_{n+1}$ is obtained by applying the events in buffer $B_n$ to the graph snapshot $G_n$. Specifically, $G_{n+1} = G_n \cup B_n$, where the union operation adds the new vertices and edges from the events in $B_n$ to the existing graph snapshot $G_n$. This process allows the system to maintain an up-to-date representation of the dynamic graph as new events are processed.

## Graph partitioning

Partition of vertex is the partition that contains the most neighbors of the vertex in the buffer. The partitioning strategy assigns the vertex to the partition that contains the most neighbors of the vertex in the buffer. The partitioner also has a replication factor, which allows it to assign a vertex to multiple partitions if there are multiple partitions that contain a similar number of neighbors of the vertex in the buffer. The partitioner has a capacity penalty, which penalizes partitions that have more vertices than the average partition size, to encourage more balanced partitions.

Formally, the score of a partition for a vertex is defined as:

$$S(P, v) = N(P, v) - \mu \cdot \max(0, |P| - \alpha \cdot (1 + \epsilon) \cdot \frac{1}{|P|} \sum_{P' \in P} |P'|)$$

where $N(P, v)$ is the number of neighbors of vertex $v$ in partition $P$ in the buffer, $\mu$ is the capacity penalty coefficient, $\alpha$ is the weight of the average partition size in the capacity penalty, $\epsilon$ is the imbalance tolerance, and $|P|$ is the size of partition $P$. The partitioner assigns the vertex to the top $k$ partitions with the highest scores, where $k$ is the replication factor. If there are multiple partitions with same scores, the partitioner randomly assigns the vertex to some of those partitions until it reaches the replication factor.

## Embedding model

Given dynamic graph (i.e., a graph that changes over time), the DynNode2Vec algorithm (Mahdavi et al. 2018) learns continuous feature representations for nodes in the graph at different time steps. The main idea behind DynNode2Vec is to extend the node2vec algorithm (Grover and Leskovec 2016) to handle dynamic graphs by incorporating temporal information into the random walk strategy and embedding learning process. The goal is to capture both the structural and temporal dynamics of the graph in the learned embeddings.

In the following listing, pseudocode for the DynNode2Vec algorithm is presented:

```
Input: Dynamic graphs G1, G2, ..., GT
Output: Embeddings Z1, Z2, ..., ZT

// t = 1
Run static node2vec on G1 to train Skip-gram_1 and obtain Z1.

For t = 2 ... T:
     // Identify evolving nodes between G_{t-1} and G_t
     Compute:
         V_add  = nodes added from t-1 to t
         E_add  = edges added from t-1 to t
         E_del  = edges deleted from t-1 to t
     delta_V_t = V_add union { v in V_t | exists (v, u) in (E_add union E_del) }

     // Evolving random walks only from changed regions
     Walk_n = node2vec_walks(G_t, start_nodes=delta_V_t, p, q, walk_length, num_walks)

     // Dynamic Skip-gram: initialize from previous time
     Initialize Skip-gram_t with weights of Skip-gram_{t-1}
     Update Skip-gram_t vocabulary for any new nodes
     Train Skip-gram_t on Walk_n
     Extract Z_t from Skip-gram_t
End For
```

In this paper, graph is represented as a sequence of events. Algorithm can be easily adapted to this representation by buffering events and applying them to the graph snapshot at each time step. The algorithm captures the evolving nature of the graph by focusing on the changed regions and updating the embeddings accordingly.

Since one vertex can be assigned to multiple partitions, each partition produces one embedding for corresponding vertex. Let $z_p = h_p(u)$ be the embedding of vertex $u$ in partition $p$. The final embedding of vertex $u$ is obtained by averaging the embeddings from all partitions that contain the vertex:

$$z_u = \frac{1}{|P_u|} \sum_{p \in P_u} z_p$$

where $P_u = \{p | u \in V_p\}$ is the set of partitions that contain vertex $u$. This averaging process allows the system to combine information from multiple partitions, potentially improving the quality of the final embedding by leveraging diverse perspectives on the vertex's relationships within the graph.

If replication factor is set to 1, then the final embedding of vertex $u$ is simply the embedding from the single partition that contains the vertex:

$$z_u = z_p$$

where $p$ is the partition that contains vertex $u$. In this case, the embedding is solely based on the information available in that single partition, which may limit the quality of the embedding if the partition does not capture sufficient context about the vertex's relationships within the graph.

Additionally, if vertex is not partitioned yet, then the final embedding of vertex $u$ is set to zero vector:

$$z_u = 0$$

## Benchmarks and evaluation metrics

To evaluate the effectiveness of the graph vertex embeddings generated in a distributed environment with community-aware partitioning, several criteria are considered:

**Embedding Quality**: The quality of the embeddings is assessed using metrics such as F1-score of reconstructed graphs (Yip et al. 2023), which measures how well the embeddings capture the relationships between vertices in the original graph. Higher F1-scores indicate better preservation of graph structure in the embeddings. Let $G = (V, E)$ be the original graph and $G' = (V, E')$ be the reconstructed graph from embeddings. $G'$ is constructed by connecting k closest vertices in the embedding space, where k is the number of edges in the original graph. The F1-score is calculated as follows:

$$F1 = 2 * \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

where

$$\text{Precision} = \frac{1}{|V|} \sum_{v \in V} \frac{|N(v) \cap N'(v)|}{|N'(v)|}$$

$$\text{Recall} = \frac{1}{|V|} \sum_{v \in V} \frac{|N(v) \cap N'(v)|}{|N(v)|}$$

and $N(v)$ and $N'(v)$ are the neighbors of vertex $v$ in the original and reconstructed graphs, respectively.

**Partition Quality**: The quality of the partitions is evaluated based on metrics such as edge cut, which measures the number of edges that connect vertices in different partitions. Lower edge cuts indicate better preservation of community structures within partitions. Edge cut is defined as:

$$\text{EdgeCut} = \frac{1}{|E|} \sum_{(u,v) \in E} \mathbb{I}(p(u) \neq p(v))$$

where $E$ is the set of edges in the original graph, $p(u)$ is the partition of vertex $u$, and $\mathbb{I}(\cdot)$ is the indicator function.

**Balance of Partitions**: The balance of the partitions is assessed by measuring the size of each partition and ensuring that they are within a bounded interval. This helps to ensure that the workload is evenly distributed across machines in the distributed environment.

**Partitioning Time**: The time taken to partition the graph is measured to evaluate the efficiency of the partitioning algorithms. Faster partitioning times are preferred, especially for large graphs, as they reduce the overall processing time in a distributed environment.

**Repartitioning amount**: The amount of repartitioning required when new vertices arrive is measured to assess the stability of the partitioning strategy. Lower amounts of repartitioning indicate that the partitioning strategy is more stable and can handle dynamic changes in the graph without significant disruption. Given two graph snapshots $G_n$ and $G_{n+1}$, the amount of repartitioning is defined as:

$$\text{RepartitioningAmount} = \frac{1}{|V_n|} \sum_{v \in V_n} \mathbb{I}(p_n(v) \neq p_{n+1}(v))$$

Where $V_n$ is the set of vertices in graph snapshot $G_n$, $p_n(v)$ is the partition of vertex $v$ in snapshot $G_n$, and $p_{n+1}(v)$ is the partition of vertex $v$ in snapshot $G_{n+1}$. This metric quantifies the proportion of vertices that have been reassigned to different partitions between two consecutive snapshots, providing insight into the stability and adaptability of the partitioning strategy in response to dynamic changes in the graph. It has to be noted that only vertices that are present in both snapshots are considered for this metric, as newly added vertices do not have a previous partition assignment to compare against and thus do not contribute to the measure of repartitioning amount.

# Results and discussion

## Partitioning hyperparameters

Throughout the experiments, the following hyperparameters are used for partitioning:

| Hyperparameter | Value |
|---|---|
| Buffer size | 1000 |
| $\mu$ | 1 |
| $\alpha$ | 1 |
| $\epsilon$ | 0.1 |
| Partitions (P) | $\{1, 2, 4, 8\}$ |
| Replication factor (RF) | $\{1, 3\}$ |

These hyperparameters are chosen to balance the trade-offs between embedding quality, partition balance, and assignment latency. In particular, the buffer size is set to 1000 to allow for sufficient information to be gathered before making partitioning decisions. For lower buffer sizes, the partitioner has less information to make informed decisions, which can lead to suboptimal partitioning and lower embedding quality. The capacity penalty coefficient ($\mu$) is set to 1 to encourage balanced partitions, while the weight of the average partition size ($\alpha$) is also set to 1 to ensure that the capacity penalty is proportional to the average partition size. The imbalance tolerance ($\epsilon$) is set to 0.1 to allow for some flexibility in partition sizes while still encouraging balance. The number of partitions (P) is varied between 1, 2, 4, and 8 to evaluate the impact of partitioning strategy on embedding quality and partition balance. Finally, the replication factor (RF) is varied between 1 and 3 to analyze the effect of replicating vertices across multiple partitions on embedding quality.

# Conclusion

# References

Adamic, Lada A, and Natalie Glance. 2005. "The Political Blogosphere and the 2004 US Election: Divided They Blog." *Proceedings of the 3rd International Workshop on Link Discovery*, 36–43.

Albert, Réka, and Albert-László Barabási. 2002. "Statistical Mechanics of Complex Networks." *Reviews of Modern Physics* 74 (1): 47.

Backstrom, Lars, Dan Huttenlocher, Jon Kleinberg, and Xiangyang Lan. 2006. "Group Formation in Large Social Networks: Membership, Growth, and

Evolution." *Proceedings of the 12th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 44–54.

Barabási, Albert-László, and Réka Albert. 1999. "Emergence of Scaling in Random Networks." *Science* 286 (5439): 509–12.

Benlic, Una, and Jin-Kao Hao. 2010. "An Effective Multilevel Memetic Algorithm for Balanced Graph Partitioning." *2010 22nd IEEE International Conference on Tools with Artificial Intelligence* 1: 121–28.

Çatalyürek, Ümit, Karen Devine, Marcelo Faraj, et al. 2023. "More Recent Advances in (Hyper) Graph Partitioning." *ACM Computing Surveys* 55 (12): 1–38.

Church, Kenneth Ward. 2017. "Word2Vec." *Natural Language Engineering* 23 (1): 155–62.

Fang, Peng, Arijit Khan, Siqiang Luo, et al. 2023. "Distributed Graph Embedding with Information-Oriented Random Walks." *arXiv Preprint arXiv:2303.15702*.

Grover, Aditya, and Jure Leskovec. 2016. "Node2vec: Scalable Feature Learning for Networks." *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 855–64.

Huang, Jiewen, and Daniel J Abadi. 2016. "Leopard: Lightweight Edge-Oriented Partitioning and Replication for Dynamic Graphs." *Proceedings of the VLDB Endowment* 9 (7).

Ji, Genlin, and Xiaohan Ling. 2007. "Ensemble Learning Based Distributed Clustering." *Emerging Technologies in Knowledge Discovery and Data Mining: PAKDD 2007 International Workshops Nanjing, China, May 22-25, 2007 Revised Selected Papers 11*, 312–21.

Krebs, Valdis E. 2002. "Mapping Networks of Terrorist Cells." *Connections* 24 (3): 43–52.

Leskovec, Jure, Daniel Huttenlocher, and Jon Kleinberg. 2010a. "Predicting Positive and Negative Links in Online Social Networks." *Proceedings of the 19th International Conference on World Wide Web*, 641–50.

Leskovec, Jure, Daniel Huttenlocher, and Jon Kleinberg. 2010b. "Signed Networks in Social Media." *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, 1361–70.

Leskovec, Jure, Kevin J Lang, Anirban Dasgupta, and Michael W Mahoney.

2009. “Community Structure in Large Networks: Natural Cluster Sizes and the Absence of Large Well-Defined Clusters.” *Internet Mathematics* 6 (1): 29–123.

Li, Mu, Li Zhou, Zichao Yang, et al. 2013. “Parameter Server for Distributed Machine Learning.” *Big Learning NIPS Workshop* 6.

Lombardo, Gianfranco, and Agostino Poggi. 2019. “A Scalable and Distributed Actor-Based Version of the Node2Vec Algorithm.” *Proceedings of the 20th Workshop From Objects to Agents* (Parma, Italy), 134–41.

Mahdavi, Sedigheh, Shima Khoshraftar, and Aijun An. 2018. “Dynnode2vec: Scalable Dynamic Network Embedding.” *2018 IEEE International Conference on Big Data (Big Data)*, 3762–65.

Nicoara, Daniel, Shahin Kamali, Khuzaima Daudjee, and Lei Chen. 2015. “Hermes: Dynamic Partitioning for Distributed Social Network Graph Databases.” *EDBT*, 25–36.

Romero Ruiz, Emmanuel, and Carlos Segura. 2018. “Memetic Algorithm with Hungarian Matching Based Crossover and Diversity Preservation.” *Computación y Sistemas* 22 (2): 347–61.

Rozemberczki, Benedek, and Rik Sarkar. 2021. *Twitch Gamers: A Dataset for Evaluating Proximity Preserving and Structural Role-Based Node Embeddings*.

Sanders, Peter, and Christian Schulz. 2011. “Engineering Multilevel Graph Partitioning Algorithms.” *European Symposium on Algorithms*, 469–80.

Sanders, Peter, and Christian Schulz. 2012. “Distributed Evolutionary Graph Partitioning.” *2012 Proceedings of the Fourteenth Workshop on Algorithm Engineering and Experiments (ALENEX)*, 16–29.

Savić, Miloš, Mirjana Ivanović, and Bojana Dimić Surla. 2017. “Analysis of Intra-Institutional Research Collaboration: A Case of a Serbian Faculty of Sciences.” *Scientometrics* 110: 195–216.

Ugander, Johan, and Lars Backstrom. 2013. “Balanced Label Propagation for Partitioning Massive Graphs.” *Proceedings of the Sixth ACM International Conference on Web Search and Data Mining*, 507–16.

Van Der Hofstad, Remco. 2024. *Random Graphs and Complex Networks*. Vol. 54. Cambridge university press.

Vaquero, Luis, Felix Cuadrado, Dionysios Logothetis, and Claudio Martella. 2013. “Adaptive Partitioning for Large-Scale Dynamic Graphs.” *Proceedings*

*of the 4th Annual Symposium on Cloud Computing*, 1–2.

Watts, Duncan J, and Steven H Strogatz. 1998. "Collective Dynamics of 'Small-World'networks." *Nature* 393 (6684): 440–42.

Xu, Ning, Lei Chen, and Bin Cui. 2014. "LogGP: A Log-Based Dynamic Graph Partitioning Method." *Proceedings of the VLDB Endowment* 7 (14): 1917–28.

Yip, Hong Yung, Chidaksh Ravuru, Neelabha Banerjee, et al. 2023. *RESTORE: Graph Embedding Assessment Through Reconstruction*. https://arxiv.org/abs/2308.14659.

Zachary, Wayne W. 1977. "An Information Flow Model for Conflict and Fission in Small Groups." *Journal of Anthropological Research* 33 (4): 452–73.